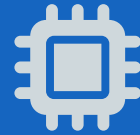


MODELAGEM COMPUTACIONAL



Autômatos Finitos Determinísticos (DFAs)

Prof. Dr. José Jairo de S. e Silva

Aula 03

Autômatos Finitos Determinísticos (DFAs)

OBJETIVOS DA AULA

🎯 Objetivos de Aprendizagem

Ao final desta aula, o estudante será capaz de:

Compreender o conceito de **autômato finito determinístico (DFA)**, sua definição formal, e utilizá-lo para reconhecer linguagens regulares.

Conteúdos:

- Motivação: do algoritmo ao autômato
- A máquina de vender chocolates como exemplo
- Definição formal de DFA (5-tupla)
- Diagramas de estados e tabelas de transição
- Função de transição estendida $\hat{\delta}$
- Aceitação e rejeição de strings
- Linguagens regulares

1. MOTIVAÇÃO

Do Algoritmo ao Autômato

Um dos objetivos deste curso é fornecer uma **definição formal para o conceito de algoritmo**. Alcançaremos isso com as **Máquinas de Turing**.

Antes disso, estudaremos um modelo mais simples: os **autômatos finitos**.

Por que estudar autômatos finitos?

- Apenas um **subconjunto** de todos os algoritmos pode ser representado por autômatos finitos (poder de expressão menor que Máquinas de Turing).
- Servem para nos **familiarizarmos com conceitos matemáticos** que serão essenciais adiante.
- Possuem **aplicações práticas** importantes:
 - Busca de padrões em textos
 - Construção de analisadores léxicos para compiladores

Hierarquia de Modelos de Computação

Podemos pensar nos modelos de computação em camadas de poder crescente:

Autômatos Finitos $\subset$ Autômatos de Pilha $\subset$ Máquinas de Turing

Cada nível reconhece uma classe maior de linguagens. Nesta aula, focaremos no primeiro nível: os **autômatos finitos determinísticos (DFAs)**.

↻ Recapitulando: o que é uma Linguagem?

Na Aula 2, definimos os conceitos de **alfabeto**, **string** e **linguagem**:

- **Alfabeto** (Σ): conjunto finito de símbolos. Ex: $\Sigma = \{0, 1\}$.
- **String**: sequência finita de símbolos de Σ . Ex: **0110**, ϵ (string vazia).
- Σ^* : conjunto de **todas** as strings possíveis sobre Σ .

Definição 2.2.1 (Aula 2): Uma **linguagem** é um conjunto $L \subseteq \Sigma^*$, ou seja, um **conjunto qualquer de strings** sobre um alfabeto.

Exemplos:

- $L_1 = \{01, 10, 111\}$ é uma linguagem finita sobre $\{0, 1\}$
- $L_2 = \{w \in \{0, 1\}^* \mid w \text{ tem número par de 1's}\}$ é uma linguagem infinita

A pergunta fundamental: dada uma string w e uma linguagem L , como **decidir** se $w \in L$?

Os **autômatos finitos** são exatamente uma forma de responder essa pergunta.

🕒 Duas Formas de Olhar para um Autômato

Autômatos finitos podem ser entendidos de **duas perspectivas** complementares:

1. Como máquina de comportamento

O foco é **descrever o funcionamento** de um sistema: uma catraca, uma porta automática, um semáforo. O autômato modela *como* o sistema reage a eventos.

2. Como reconhecedor de linguagem

O foco é **decidir se uma string pertence ou não a uma linguagem**. Dada uma sequência de símbolos, o autômato processa símbolo por símbolo e, no final, responde **SIM** (aceita) ou **NÃO** (rejeita).

Na prática, é a **mesma estrutura matemática** usada para dois propósitos diferentes. Mas em ambos os casos, a pergunta fundamental é sempre: **esta sequência de entrada é aceita ou não?**

💡 O que é um Autômato Finito, Intuitivamente?

Um autômato finito é uma **máquina de estados** que lê entradas uma por uma e muda de estado conforme **regras fixas**.

Conceito	Significado
Estado	Onde a máquina está agora
Entrada	O que acontece neste instante
Transição	Para onde a máquina vai
Estado final	Situação de aceitação

Exemplos do dia-a-dia:

- **Porta automática:** a string **FRENTE · NENHUM** pertence à linguagem das sequências que deixam a porta aberta?
- **Palavra binária:** a string **0110** pertence à linguagem das palavras com número par de 1's?

O ponto central: autômato finito **não é algo abstrato demais**. É apenas uma forma organizada de **verificar se uma string pertence a uma linguagem**, processando-a símbolo por símbolo.

2. UM EXEMPLO MOTIVADOR

A Máquina de Vender Chocolates

Considere uma máquina que vende chocolates custando £6, aceitando moedas de £1 e £2.

A máquina libera o chocolate quando a soma das moedas inseridas atinge pelo menos £6.

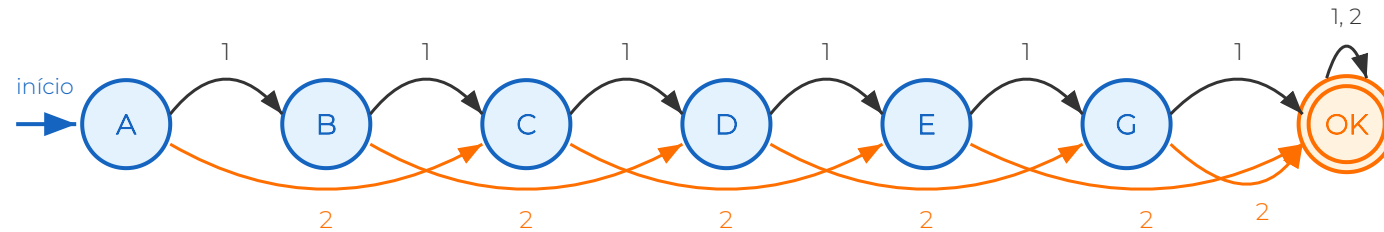
A entrada é uma sequência de moedas (uma "string" do alfabeto {1, 2}).

Pergunta central: dada uma string de moedas, ela pertence à linguagem das sequências que liberam o chocolate?

Em outras palavras: queremos verificar se a string fornecida é aceita ou rejeitada pela máquina.

Diagrama da Máquina de Chocolates

O comportamento interno da máquina pode ser descrito pelo seguinte **diagrama de estados**:



A máquina atinge o estado **OK** se a soma das moedas for ≥ 6 . O estado **A** é o estado inicial.

Entendendo o Diagrama

O diagrama anterior é chamado de **máquina de estados** ou **autômato finito determinístico**.

Como ler o diagrama:

- O **estado A** é marcado com "início", é o estado inicial da máquina.
- A partir de cada estado, a máquina faz uma **transição** para cada moeda fornecida.
- A máquina libera o chocolate se, e somente se, atingir o estado **OK**.
- Estamos num cenário simplificado: a máquina **não dá troco**.

Exemplo: A string $w = 122$ pertence à linguagem?

$$A \xrightarrow{1} B \xrightarrow{2} D \xrightarrow{2} G$$

A máquina para em G (não é estado final). A string **122 não pertence** à linguagem. Chocolate não liberado.

A string $w = 222$ pertence? $A \xrightarrow{2} C \xrightarrow{2} E \xrightarrow{2} OK \in F$. **Sim, pertence!**

Q Dissecando a Máquina de Chocolates

Olhando para o diagrama, conseguimos identificar **cinco ingredientes** que descrevem completamente a máquina:

1. **As situações possíveis** (os "estados"):

$$\{A, B, C, D, E, G, OK\}$$

São as **7 configurações** em que a máquina pode estar. Esse conjunto é **finito**.

2. **O que a máquina pode receber** (o "alfabeto"):

$$\{1, 2\}$$

São os **tipos de entrada** que a máquina aceita: moedas de £1 ou £2.

3. **As regras de transição** (o "manual de instruções"):

Para cada combinação de estado + entrada, existe **exatamente uma** resposta. Exemplo: se está em **D** e recebe **2**, vai para **G**. Não há ambiguidade.

Q Dissecando a Máquina de Chocolates (cont.)

4. A situação inicial (o "ponto de partida"):

A

É o estado em que a máquina se encontra **antes de receber qualquer entrada**. Toda string começa sendo processada a partir daqui.

5. As situações de aceitação (os estados "bons"):

$\{OK\}$

São os estados que representam **sucesso**: se a máquina termina aqui após ler toda a string, a resposta é **SIM**, a string pertence à linguagem. Se termina em qualquer outro estado, a resposta é **NÃO**.

Perceba: com esses **cinco ingredientes**, conseguimos reconstruir completamente o comportamento da máquina. Não precisamos de mais nada. Guarde essa observação!

Outro Exemplo Físico: Porta Automática

Considere uma **porta automática** (como as de supermercados). Ela possui sensores na frente e atrás:

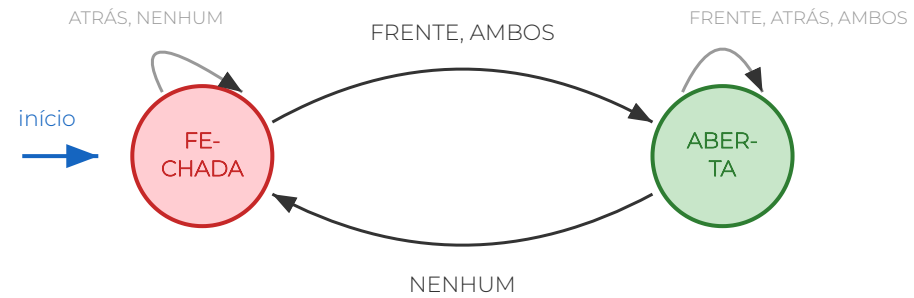
- Está inicialmente **fechada**.
- Quando alguém se aproxima pela **frente**, a porta **abre**.
- A porta permanece **aberta** enquanto houver alguém na frente **ou** atrás.
- Quando **ninguém** está próximo, a porta **fecha**.

Entradas do sensor: $\Sigma = \{\text{FRENTE, ATRÁS, AMBOS, NENHUM}\}$

Estados: $Q = \{\text{FECHADA, ABERTA}\}$

Pergunta: dada uma string de leituras dos sensores, a porta **termina aberta** (string pertence à linguagem) ou **fechada** (não pertence)?

Diagrama da Porta Automática



Q Dissecando a Porta Automática

Vamos identificar os **mesmos cinco ingredientes** que encontramos na máquina de chocolates:

1. **As situações possíveis** (os "estados"):

{FECHADA, ABERTA}

Apenas **2 estados**. A porta está em um deles a cada instante.

2. **O que a porta pode receber** (o "alfabeto"):

{FRENTE, ATRÁS, AMBOS, NENHUM}

São as **leituras dos sensores**: quem está próximo da porta neste instante.

3. **As regras de transição**:

Para cada estado + leitura, há **exatamente uma** resposta. Exemplo: se está **FECHADA** e recebe **FRENTE**, vai para **ABERTA**. Se está **ABERTA** e recebe **NENHUM**, vai para **FECHADA**.

Q Dissecando a Porta Automática (cont.)

4. A situação inicial:

FECHADA

A porta começa fechada. Toda string é processada a partir deste estado.

5. As situações de aceitação:

{ABERTA}

Se, após ler toda a string de leituras, a porta termina **aberta**, a string **pertence** à linguagem. Se termina **fechada**, **não pertence**.

Verificação: A string $w = \text{FRENTE} \cdot \text{ATRÁS} \cdot \text{NENHUM}$ pertence à linguagem?

$$\text{FECHADA} \xrightarrow{\text{FRENTE}} \text{ABERTA} \xrightarrow{\text{ATRÁS}} \text{ABERTA} \xrightarrow{\text{NENHUM}} \text{FECHADA} \notin F$$

A string **não pertence**. E $w = \text{FRENTE} \cdot \text{ATRÁS}$? $\text{FECHADA} \xrightarrow{\text{FR.}} \text{ABERTA} \xrightarrow{\text{AT.}} \text{ABERTA} \in F$.
Pertence!

O Padrão dos Cinco Ingredientes

Nos dois exemplos encontramos **exatamente os mesmos cinco ingredientes**:

#	Máquina de Chocolates	Porta Automática
1	$\{A, B, C, D, E, G, OK\}$	$\{\text{FECHADA}, \text{ABERTA}\}$
2	$\{1, 2\}$	$\{\text{FR}, \text{AT}, \text{AM}, \text{NE}\}$
3	regras (tabela/diagrama)	regras (tabela/diagrama)
4	A	FECHADA
5	$\{OK\}$	$\{\text{ABERTA}\}$

Isso **não é coincidência**. Todo autômato finito é descrito por esses cinco ingredientes.

Na próxima seção, vamos dar **nomes matemáticos** a cada ingrediente e formalizar essa estrutura como uma **5-tupla**.

3. DEFINIÇÃO FORMAL DE DFA

Definição 3.1.1: Autômato Finito Determinístico (DFA)

Definição 3.1.1: Um **Autômato Finito Determinístico** é uma 5-tupla $D = (Q, \Sigma, \delta, q_0, F)$, tal que:

Componente	Descrição
Q	conjunto finito de estados
Σ	conjunto de símbolos de entrada (alfabeto)
δ	função de transição $\delta : Q \times \Sigma \rightarrow Q$
$q_0 \in Q$	estado inicial
$F \subseteq Q$	conjunto de estados finais (de aceitação)

A sigla **DFA** vem do inglês *Deterministic Finite Automaton*.

☰ Entendendo a Definição

A definição é **genérica**: ela descreve a estrutura de qualquer DFA.

Para obter um DFA **específico**, precisamos dar uma **instância** da 5-tupla, atribuindo valores concretos a cada componente.

É como a diferença entre a definição geral de "função" e uma função particular como $f(x) = x^2$.

Pontos-chave:

- δ é uma **função total**: para cada par (estado, símbolo), existe exatamente **um** próximo estado.
- É isso que torna o autômato **determinístico**: não há ambiguidade sobre o próximo estado.
- O conjunto F pode ser vazio (nenhuma string é aceita) ou igual a Q (toda string é aceita).

Exemplo 3.1: Máquina de Vender Chocolates

A definição formal do DFA correspondente à máquina de chocolates é $C = (Q_c, \Sigma_c, \delta_c, A, F_c)$:

- $Q_c = \{A, B, C, D, E, G, OK\}$
- $\Sigma_c = \{1, 2\}$
- δ_c é definida caso a caso:

	1	2
$\rightarrow A$	B	C
B	C	D
C	D	E
D	E	G
E	G	OK
G	OK	OK
$*OK$	OK	OK

- Estado inicial: A | Estados finais: $F_c = \{OK\}$

Como Ler a Tabela de Transição

A tabela do slide anterior é uma forma compacta de representar a função δ_c . Veja como interpretá-la:

- Cada **linha** corresponde a um estado do autômato.
- Cada **coluna** corresponde a um símbolo de entrada (**1** ou **2**).
- O valor na **célula** é o estado resultante ao aplicar δ_c .

Exemplo de leitura: Na linha **D**, coluna **2**, encontramos **G**. Isso significa:

$$\delta_c(D, 2) = G$$

ou seja, se a máquina está no estado **D** e recebe uma moeda de £2, ela transita para o estado **G**.

Convenções da tabela:

- $\rightarrow$ antes de um estado indica o **estado inicial** (ex: $\rightarrow A$).
- $*$ antes de um estado indica um **estado final** (ex: $*OK$).

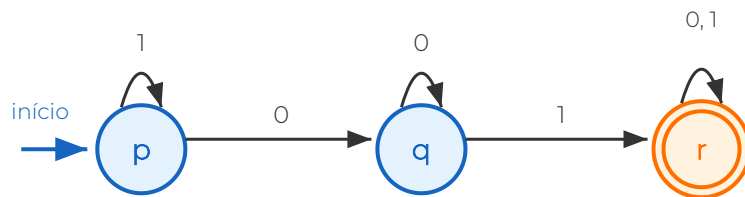
4. DIAGRAMAS E TABELAS DE TRANSIÇÃO

Exercício Resolvido 3.1

Enunciado: Desenhe o diagrama do autômato $A = (Q, \Sigma, \delta, p, \{r\})$, com $Q = \{p, q, r\}$, $\Sigma = \{0, 1\}$ e:

$$\delta(p, 0) = q, \quad \delta(p, 1) = p, \quad \delta(q, 0) = q, \quad \delta(q, 1) = r, \quad \delta(r, 0) = r, \quad \delta(r, 1) = r$$

Solução:



Este DFA aceita strings sobre $\{0, 1\}$ que **contêm a substring 01**.

Tabela de Transição

A função δ pode ser representada como uma **tabela**, facilitando a leitura:

	0	1
$\rightarrow p$	q	r
$*r$	r	r

Convenções:

- A seta $\rightarrow$ indica o **estado inicial**.
- O asterisco $*$ indica um **estado final** (de aceitação).
- Cada célula contém o resultado de $\delta(\text{estado}, \text{símbolo})$.

No decorrer do curso, ao nos referirmos a um DFA, poderemos apresentar a 5-tupla, a tabela de transições ou o diagrama, que são **representações equivalentes**.

5. ACEITAÇÃO E REJEIÇÃO DE STRINGS

⚙️ A Ideia Intuitiva

Um DFA processa uma string $w = w_1w_2 \dots w_n$ da seguinte forma:

1. Começa no **estado inicial** q_0 .
2. Lê o primeiro símbolo w_1 e faz uma **transição** de acordo com δ .
3. Repete para cada símbolo subsequente $w_2, w_3, \dots, w_n$.
4. Após processar o último símbolo, verifica em qual estado parou.

A pergunta central:

- | Em que estado o DFA se encontra após processar o último símbolo de w ?
 - Se parou em um **estado final** $q \in F$: o DFA **aceita** w .
 - Se parou em um **estado não-final** $q \notin F$: o DFA **rejeita** w .

Exemplo: Processando uma String

Considere o DFA do Exercício Resolvido 3.1 e a string $w = 10011$:

$$p \xrightarrow{1} p \xrightarrow{0} q \xrightarrow{0} q \xrightarrow{1} r \xrightarrow{1} r$$

O DFA parou no estado r , que é estado final ($r \in F$).

Resultado: O DFA **aceita** a string 10011.

Agora considere $w = 111$:

$$p \xrightarrow{1} p \xrightarrow{1} p \xrightarrow{1} p$$

O DFA parou no estado p , que **não** é estado final ($p \notin F$).

Resultado: O DFA **rejeita** a string 111.

Definição 3.1.2: Função de Transição Estendida

Para formalizar a ideia de processar uma **string inteira** (não apenas um símbolo), precisamos estender δ .

Definição 3.1.2: Dado um DFA $D = (Q, \Sigma, \delta, q_0, F)$, a **função de transição estendida** $\hat{\delta} : Q \times \Sigma^* \rightarrow Q$ é definida indutivamente:

Caso base ($w = \varepsilon$):

$$\hat{\delta}(q, \varepsilon) = q$$

Passo indutivo ($|w| > 0$): Seja $w = xa$, onde $x \in \Sigma^*$ e $a \in \Sigma$. Então:

$$\hat{\delta}(q, w) = \delta(\hat{\delta}(q, x), a)$$

Intuição: $\hat{\delta}(q, w)$ nos diz em qual estado o DFA estará após processar toda a string w , partindo do estado q .

Exemplo: Calculando $\hat{\delta}$

Usando o DFA do Exercício Resolvido 3.1, vamos calcular $\hat{\delta}(p, 01)$:

$$\hat{\delta}(p, 01) = \delta(\hat{\delta}(p, 0), 1)$$

Precisamos de $\hat{\delta}(p, 0)$:

$$\hat{\delta}(p, 0) = \delta(\hat{\delta}(p, \varepsilon), 0) = \delta(p, 0) = q$$

Portanto:

$$\hat{\delta}(p, 01) = \delta(q, 1) = r$$

Como $r \in F$, o DFA **aceita** a string **01**.

 Exemplo: Calculando $\hat{\delta}$ para uma string maior

Agora vamos calcular $\hat{\delta}(p, 100)$. Descascamos a string da direita para a esquerda:

$$\hat{\delta}(p, 100) = \delta(\hat{\delta}(p, 10), 0)$$

Precisamos de $\hat{\delta}(p, 10)$:

$$\hat{\delta}(p, 10) = \delta(\hat{\delta}(p, 1), 0)$$

Precisamos de $\hat{\delta}(p, 1)$:

$$\hat{\delta}(p, 1) = \delta(\hat{\delta}(p, \varepsilon), 1) = \delta(p, 1) = p$$

Como $q \notin F$, o DFA **rejeita** a string **100**. De fato, **100** não contém a substring **01**.

Definição 3.1.3: Aceitação e Rejeição

Definição 3.1.3: Seja $D = (Q, \Sigma, \delta, q_0, F)$ um DFA e $w \in \Sigma^*$.

- Se $\hat{\delta}(q_0, w) \in F$, dizemos que D **aceita** a string w .
- Se $\hat{\delta}(q_0, w) \notin F$, dizemos que D **rejeita** a string w .

Em palavras: o DFA aceita w se, ao processar w a partir do estado inicial, termina em um estado final.

Definição 3.1.4: Linguagem de um DFA

Definição 3.1.4: A **linguagem de um DFA** D , denotada por $L(D)$, é o conjunto de todas as strings aceitas por D :

$$L(D) = \{w \in \Sigma^* \mid \hat{\delta}(q_0, w) \in F\}$$

Se L é a linguagem de um DFA D , dizemos que D **decide** (ou **aceita**) a linguagem L .

Exemplo: O DFA do Exercício Resolvido 3.1 aceita a linguagem:

$$L(A) = \{w \in \{0, 1\}^* \mid w \text{ contém a substring } 01\}$$

Detalhando: Quais strings pertencem a $L(A)$?

Testando strings no DFA A (p = "não vi 0", q = "vi 0", r = "vi 01"):

String w	Caminho	Resultado
01	$p \rightarrow q \rightarrow r \in F$	Aceita
101	$p \rightarrow p \rightarrow q \rightarrow r \in F$	Aceita
111	$p \rightarrow p \rightarrow p \rightarrow p \notin F$	Rejeita
000	$p \rightarrow q \rightarrow q \rightarrow q \notin F$	Rejeita

Padrão: strings com **01** em alguma posição são aceitas. Strings só de 1's ou só de 0's são rejeitadas.

$L(A)$ é o **conjunto infinito** de todas as strings sobre $\{0, 1\}$ que possuem **01** como substring. O DFA verifica isso símbolo por símbolo, sem precisar "olhar para trás".

6. LINGUAGENS REGULARES

Definição 3.1.5: Linguagem Regular

Definição 3.1.5: Uma linguagem L é dita **regular** se existe um DFA D tal que $L = L(D)$.

Em outras palavras: L é regular se **conseguimos construir um DFA** que aceita exatamente as strings de L e rejeita todas as demais.

Atenção ao "se existe": não precisamos encontrar o DFA mais eficiente ou mais bonito. Basta **existir algum** DFA que funcione.

Linguagens Regulares: Exemplos

Já vimos vários DFAs ao longo da aula. Cada um deles **prova** que a linguagem correspondente é regular:

Linguagem	DFA que a reconhece	Regular?
Strings que liberam o chocolate (soma ≥ 6)	Máquina de chocolates (7 estados)	Sim
Strings contendo a substring 01	Exercício Resolvido 3.1 (3 estados)	Sim
Strings de leituras que deixam a porta aberta	Porta automática (2 estados)	Sim

E linguagens não regulares? Existem linguagens para as quais **nenhum DFA é capaz** de decidir a pertinência. Veremos um exemplo a seguir.

A pergunta central do estudo de autômatos é: **quais linguagens são regulares e quais não são?**

⚠ Por que $\{a^n b^n\}$ não é regular?

Considere $L = \{a^n b^n \mid n \geq 0\} = \{\varepsilon, ab, aabb, aaabbb, \dots\}$

Imagine uma pessoa que lê a string da esquerda para a direita, **um símbolo por vez**. Para decidir se a string pertence a L , ela precisa contar os a 's e depois verificar se há a mesma quantidade de b 's.

Mas essa pessoa tem uma **limitação**: ela só pode usar uma folha de papel com espaço para anotar **uma única palavra** (o estado atual). Ela não pode escrever números.

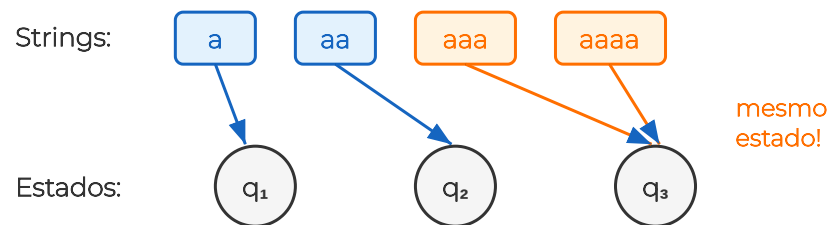
Ela lê...	Ela precisaria anotar...
ab	"vi 1 a " $\rightarrow$ confere 1 b ✓
$aabb$	"vi 2 a 's" $\rightarrow$ confere 2 b 's ✓
$a^{100}b^{100}$	"vi 100 a 's" $\rightarrow$ confere 100 b 's ✓

O problema é que a folha tem espaço **limitado** (finitos estados). Com k estados, ela consegue distinguir no máximo k quantidades diferentes. Mas n pode ser $1, 2, 3, \dots, 1000, \dots$, infinitas possibilidades!

⚠ Por que $\{a^n b^n\}$ não é regular? (cont.)

Se o DFA tem apenas **3** estados, ele precisa dar conta de $a, aa, aaa, aaaa, \dots$

Com 3 anotações possíveis e 4 strings, **duas vão receber a mesma anotação** (mesmo estado):



Se aaa e $aaaa$ levam ao **mesmo estado**, o DFA não sabe qual das duas leu. Então:

- Ele precisa aceitar $aaabbb$ (pois $aaabbb \in L$, com 3 a 's e 3 b 's).
- Mas como $aaaa$ leva ao mesmo estado, ele também aceita $aaaabbb$.
- Porém $aaaabbb \notin L$, pois $4 \neq 3$. **Contradição!**

Com k estados e $k + 1$ strings, sempre haverá esse conflito. Portanto:

Nenhum DFA consegue reconhecer $\{a^n b^n\}$. Essa linguagem **não é regular**. A prova formal usa o **Lema do Bombeamento**, que veremos em aulas futuras.

7. PROJETANDO AUTÔMATOS

💡 Projetar Autômatos é um Processo Criativo

Construir um DFA para uma linguagem dada é uma tarefa que exige **criatividade** e **experiência**. Não há um algoritmo mecânico para isso.

Estratégia geral:

1. **Identifique a informação essencial** que precisa ser rastreada para decidir se a string pertence à linguagem.
2. **Crie estados** que representem essa informação. Cada estado deve codificar o que é relevante sobre a entrada lida até o momento.
3. **Defina as transições** pensando: "se estou no estado q e leio o símbolo a , como a informação rastreada se atualiza?"

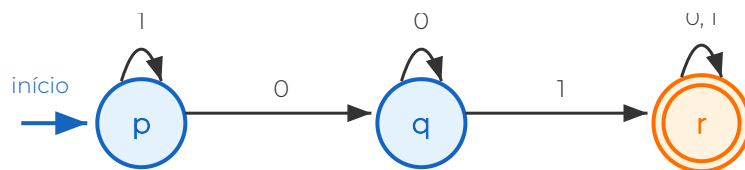
Dica: Pense nos estados como "resumos" do que foi lido. O DFA não precisa lembrar da string inteira, apenas da **informação necessária** para a decisão final.

Exemplo: Para decidir se uma string termina em **00**, basta lembrar os **dois últimos símbolos** lidos.

Exercício Resolvido 3.2: Strings com Substring 01

Enunciado: Seja $\Sigma = \{0, 1\}$. Construa um DFA para a linguagem $L = \{w \mid w \text{ contém a substring } 01\}$.

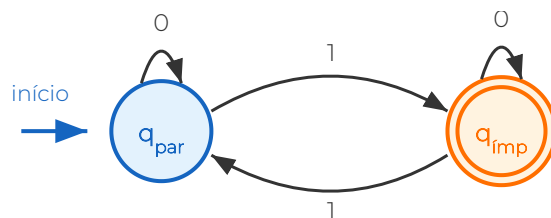
Solução: O DFA do Exercício Resolvido 3.1 serve!



Ideia: estado p = "ainda não vi 0", estado q = "vi 0 mas não 01", estado r = "vi 01".

Exemplo: DFA para Número Ímpar de 1's

Enunciado: Construa um DFA para $L = \{w \in \{0, 1\}^* \mid w \text{ tem número ímpar de 1's}\}$.



Ideia: O DFA usa dois estados para rastrear a **paridade** do número de 1's lidos.

- q_{par} : número par de 1's vistos até agora (estado inicial, pois zero é par)
- q_{imp} : número ímpar de 1's vistos (estado final)

Entendendo as Transições do DFA "Ímpar de 1's"

Por que ler 0 mantém o estado? O símbolo **0** não é um **1**, então a contagem de 1's não muda. A paridade continua a mesma, e o DFA fica onde está.

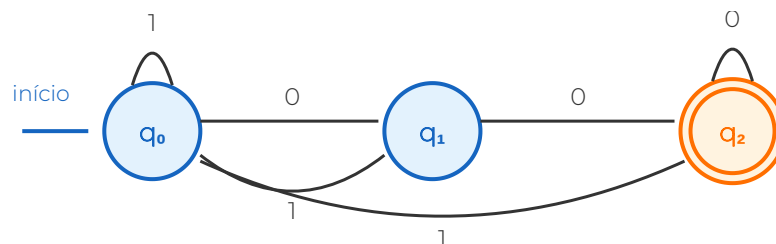
Por que ler 1 troca de estado? Cada novo **1** inverte a paridade: par $\rightarrow$ ímpar, ímpar $\rightarrow$ par. Por isso $q_{par} \xrightarrow{1} q_{ímp} \in q_{ímp} \xrightarrow{1} q_{par}$.

Exemplos:

- $w = 10$: $q_{par} \xrightarrow{1} q_{ímp} \xrightarrow{0} q_{ímp} \in F$. Um 1 (ímpar), **aceita** ✓
- $w = 1010$: $q_{par} \xrightarrow{1} q_{ímp} \xrightarrow{0} q_{ímp} \xrightarrow{1} q_{par} \xrightarrow{0} q_{par} \notin F$. Dois 1's (par), **rejeita** ✓
- $w = 111$: $q_{par} \xrightarrow{1} q_{ímp} \xrightarrow{1} q_{par} \xrightarrow{1} q_{ímp} \in F$. Três 1's (ímpar), **aceita** ✓

Exemplo: DFA para Strings Terminadas em 00

Enunciado: Construa um DFA para $L = \{w \in \{0, 1\}^* \mid w \text{ é terminada em } 00\}$.



- q_0 : nenhum **0** recente no final; q_1 : último símbolo foi **0**; q_2 : últimos dois símbolos foram **00**.

Por que q_2 volta para q_0 ao ler 1? Porque a linguagem exige que a string **termine** em **00**. Se após dois 0's aparece um **1**, o final da string já não é mais **00**, e o DFA precisa recomençar a busca.

Exemplo: $w = 001$: $q_0 \xrightarrow{0} q_1 \xrightarrow{0} q_2 \xrightarrow{1} q_0 \notin F$. Apesar de ter **00**, a string **termina em 1**, então não pertence a L .

8. CONSTRUÇÃO DO PRODUTO

Construção do Produto de DFAs

Dados dois DFAs $D_Q = (Q, \Sigma, \delta_Q, q_0, F_Q)$ e $D_P = (P, \Sigma, \delta_P, p_0, F_P)$, podemos construir um terceiro DFA D_A que **simula ambos ao mesmo tempo**.

Construção: $D_A = (A, \Sigma, \delta_A, a_0, F_A)$, onde:

Componente	Definição
A	$Q \times P$ (pares de estados)
a_0	(q_0, p_0)
δ_A	$\delta_A((q_i, p_r), s) = (\delta_Q(q_i, s), \delta_P(p_r, s))$
F_A	$\{(q_i, p_j) \mid q_i \in F_Q \text{ e } p_j \in F_P\}$

Resultado: $L(D_A) = L(D_Q) \cap L(D_P)$

A linguagem aceita por D_A é a **interseção** das linguagens de D_Q e D_P .

💡 Intuição da Construção do Produto

Por que funciona?

Na terceira transição, se D_Q estiver em q_1 e D_P estiver em p_5 , então D_A estará em (q_1, p_5) .

O DFA D_A **rastreia simultaneamente** o estado de ambos os DFAs originais.

Consequência importante:

▮ Se L_1 e L_2 são linguagens regulares, então $L_1 \cap L_2$ também é regular.

A classe das linguagens regulares é **fechada sob interseção**.

Exemplo: Produto dos DFAs Anteriores

Vamos combinar os dois DFAs que acabamos de ver:

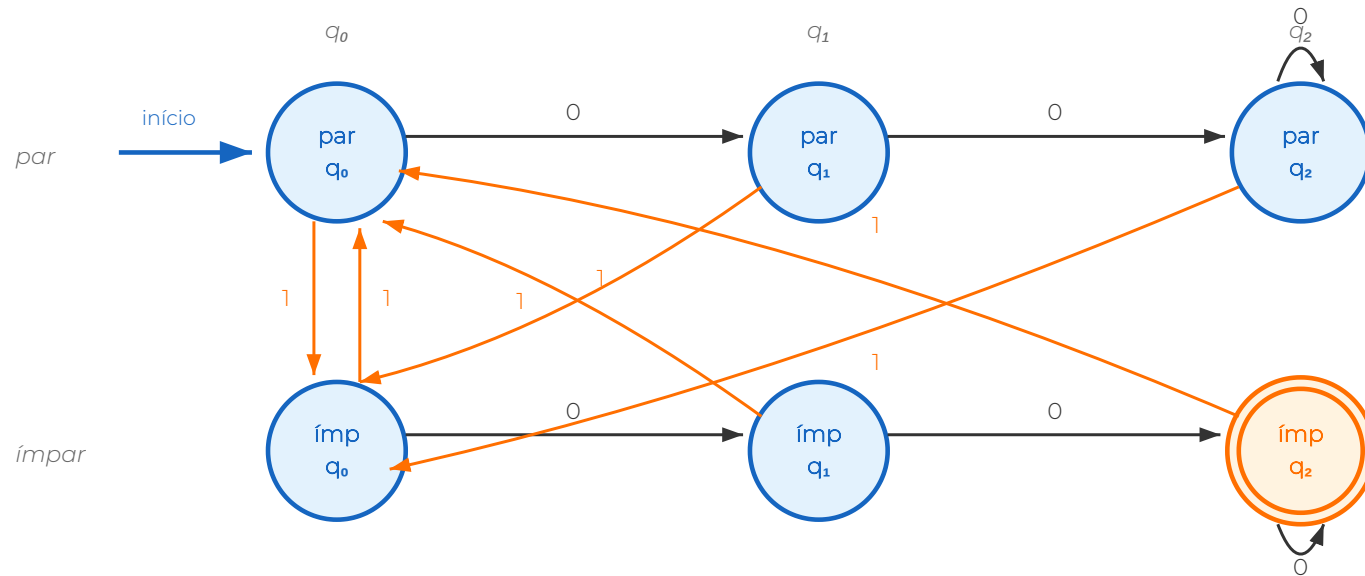
- D_1 : aceita strings com **número ímpar de 1's**, com estados $\{q_{par}, q_{ímp}\}$
- D_2 : aceita strings **terminadas em 00**, com estados $\{q_0, q_1, q_2\}$

O produto D_A tem $2 \times 3 = 6$ estados (pares):

$$(q_{par}, q_0), (q_{par}, q_1), (q_{par}, q_2), (q_{ímp}, q_0), (q_{ímp}, q_1), (q_{ímp}, q_2)$$

- **Estado inicial:** (q_{par}, q_0)
- **Estado final:** $(q_{ímp}, q_2)$, o único par onde D_1 aceita **e** D_2 aceita

Diagrama do DFA Produto



Preto = transições com **0** (paridade não muda, avança em D_2). **Laranja** = transições com **1** (paridade troca, D_2 volta a q_0).

Q Como Ler o Diagrama do Produto

O diagrama tem **duas linhas** e **três colunas**:

- **Linhas:** representam a paridade do número de 1's (D_1). Linha de cima = par, linha de baixo = ímpar.
- **Colunas:** representam o progresso em direção a terminar em **00** (D_2). q_0 = nenhum 0 recente, q_1 = último foi 0, q_2 = últimos dois foram 00.

Ao ler 0: a paridade **não muda** (fica na mesma linha), e o progresso avança para a direita ($q_0 \rightarrow q_1 \rightarrow q_2 \rightarrow q_2$). Por isso as setas pretas são **horizontais**.

Ao ler 1: a paridade **troca** (muda de linha), e o progresso **reseta** (volta a q_0 , pois o último símbolo não é 0). Por isso as setas laranja vão **na diagonal** para a coluna q_0 da outra linha.

Estado final: apenas (*ímp*, q_2) tem duplo círculo, pois é o único estado onde D_1 aceita (ímpar) e D_2 aceita (termina em 00) ao mesmo tempo.

 Exemplo: Verificando o Produto

Teste com $w = 100$: pertence a $L(D_1) \cap L(D_2)$?

Vamos acompanhar o caminho no diagrama do produto:

Símbolo	Estado em D_1	Estado em D_2	Estado no produto	O que aconteceu
início	q_{par}	q_0	(par, q_0)	ponto de partida
1	q_{imp}	q_0	(imp, q_0)	paridade trocou, progresso resetou
0	q_{imp}	q_1	(imp, q_1)	paridade manteve, avançou para q_1
0	q_{imp}	q_2	$(imp, q_2) \in F_A$	paridade manteve, chegou a q_2

Resultado: chegamos em (imp, q_2) , que é estado final. A string **100** tem número ímpar de 1's e termina em **00**. **Pertence** a $L(D_1) \cap L(D_2)$.

E $w = 110$? $(par, q_0) \xrightarrow{1} (imp, q_0) \xrightarrow{1} (par, q_0) \xrightarrow{0} (par, q_1) \notin F_A$. **Não pertence**, pois a paridade é par (dois 1's) e não termina em **00**.

💡 Complemento de Linguagens Regulares

| Se L é regular, então $\overline{L} = \Sigma^* \setminus L$ também é regular.

Prova (ideia): Seja $D = (Q, \Sigma, \delta, q_0, F)$ um DFA com $L(D) = L$.

Basta construir $D' = (Q, \Sigma, \delta, q_0, Q \setminus F)$, ou seja, **invertemos** estados finais e não-finais.

Uma string w é aceita por D' se e somente se é rejeitada por D , logo $L(D') = \overline{L}$.

☰ Propriedades de Fechamento das Linguagens Regulares

As linguagens regulares são **fechadas** sob diversas operações. Isto significa que, ao aplicar qualquer uma dessas operações sobre linguagens regulares, o resultado é também regular:

Operação	Se L_1, L_2 são regulares, então...
União	$L_1 \cup L_2$ é regular
Interseção	$L_1 \cap L_2$ é regular
Complemento	$\overline{L_1}$ é regular
Concatenação	$L_1 \cdot L_2$ é regular
Estrela de Kleene	L_1^* é regular

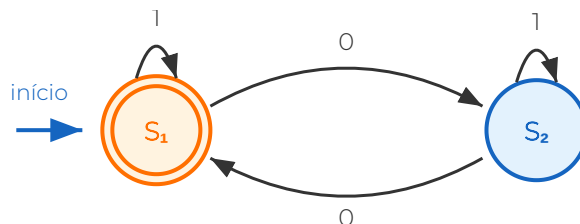
A prova do fechamento sob interseção usa a **construção do produto**. O fechamento sob complemento usa a **troca de estados finais**. As demais provas serão vistas adiante.

9. MAIS EXEMPLOS DE DFA_s

 Exemplo: DFA para Número Par de 0's

Enunciado: Construa um DFA M para $L = \{w \in \{0, 1\}^* \mid w \text{ contém número par de 0's}\}$.

Solução: $M = (\{S_1, S_2\}, \{0, 1\}, \delta, S_1, \{S_1\})$



- S_1 = número **par** de 0's lidos (estado inicial e final, pois zero é par)
- S_2 = número **ímpar** de 0's lidos
- Ler **0** troca de estado (muda a paridade); ler **1** mantém (não afeta a contagem de 0's)

Exemplo Resolvido: DFA para Múltiplos de 3 em Binário

Enunciado: Construa um DFA para $L_3 = \{w \in \{0, 1\}^* \mid N(w) \text{ é múltiplo de } 3\}$.

Ideia: O DFA lê a string da **esquerda para a direita**, um símbolo por vez. Vamos acompanhar como o número representado cresce conforme lemos a string **1011**:

Símbolos lidos	Número em binário	Valor em decimal
1	1	1
1, 0	10	2
1, 0, 1	101	5
1, 0, 1, 1	1011	11

Múltiplos de 3: O Padrão

Perceba o padrão: a cada novo símbolo lido, o valor anterior **dobra** e **soma** o novo símbolo:

$$1 \xrightarrow{\text{lê } 0} 2 \times 1 + 0 = 2 \xrightarrow{\text{lê } 1} 2 \times 2 + 1 = 5 \xrightarrow{\text{lê } 1} 2 \times 5 + 1 = 11$$

Por que dobra? É a mesma coisa que acontece em decimal. Se você tem o número **34** e escreve um **7** à direita, o resultado é:

$$347 = 10 \times 34 + 7$$

Os dígitos anteriores são multiplicados pela **base** (**10** em decimal) e o novo dígito é somado. Em binário, a base é **2**:

$$n_{\text{novo}} = 2 \times n_{\text{anterior}} + \text{símbolo lido}$$

Múltiplos de 3: A Sacada do Resto

Para verificar se um número é múltiplo de 3, não precisamos saber o número inteiro. Basta saber o **resto da divisão por 3**.

E o melhor: o novo resto depende **apenas** do resto anterior!

$$\text{resto}_{\text{novo}} = (2 \times \text{resto}_{\text{anterior}} + \text{símbolo}) \bmod 3$$

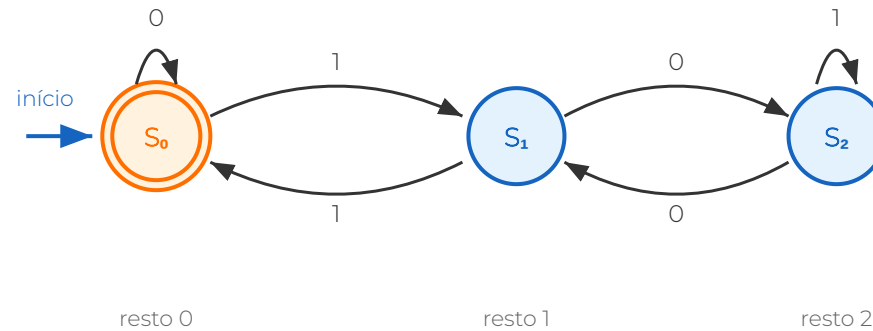
Exemplo com a string 1011:

Símbolo lido	Conta	Resto
(início)		0
1	$(2 \times 0 + 1) \bmod 3 = 1$	1
0	$(2 \times 1 + 0) \bmod 3 = 2$	2
1	$(2 \times 2 + 1) \bmod 3 = 2$	2
1	$(2 \times 2 + 1) \bmod 3 = 2$	2

Resto final = $2 \neq 0$, então **1011** (que vale **11**) **não** é múltiplo de 3. O DFA **rejeita**.

Múltiplos de 3: Diagrama

Três estados, um para cada resto possível na divisão por 3:



Múltiplos de 3: Entendendo as Transições

Cada estado representa um **resto**: S_0 (resto 0), S_1 (resto 1), S_2 (resto 2). A fórmula $(2n + b) \bmod 3$ determina todas as transições:

Estado atual (resto n)	Lê bit b	Novo resto: $(2n + b) \bmod 3$	Vai para
S_0 (resto 0)	0	$(0 + 0) \bmod 3 = 0$	S_0
S_0 (resto 0)	1	$(0 + 1) \bmod 3 = 1$	S_1
S_1 (resto 1)	0	$(2 + 0) \bmod 3 = 2$	S_2
S_1 (resto 1)	1	$(2 + 1) \bmod 3 = 0$	S_0
S_2 (resto 2)	0	$(4 + 0) \bmod 3 = 1$	S_1
S_2 (resto 2)	1	$(4 + 1) \bmod 3 = 2$	S_2

 Múltiplos de 3: Exemplo com $w = 110$ (vale 6)

Vamos acompanhar o DFA lendo **110** símbolo a símbolo:

Passo	Símbolo lido	Número formado	Valor decimal	Resto $\div 3$	Estado
início			0	0	S_0
1	1	1	1	1	S_1
2	1	11	3	0	S_0
3	0	110	6	0	S_0

Estado final: $S_0 \in F$. O número **6** é múltiplo de 3. O DFA **aceita**. ✓

O DFA não precisou saber que o número era **6**. Ele apenas rastreou o **resto** ($0 \rightarrow 1 \rightarrow 0 \rightarrow 0$) e verificou que terminou em **0**.

Múltiplos de 3: Definição Formal

Definição formal: $D = (\{S_0, S_1, S_2\}, \{0, 1\}, \delta, S_0, \{S_0\})$

	0	1
$\rightarrow *S_0$	S_0	S_1
S_1	S_2	S_0
S_2	S_1	S_2

Confira que cada entrada da tabela corresponde à fórmula do slide "Entendendo as Transições".

 Múltiplos de 3: Verificações

String 101 (vale 5, não é múltiplo de 3):

$$S_0 \xrightarrow{1} S_1 \xrightarrow{0} S_2 \xrightarrow{1} S_2 \notin F \quad \checkmark$$

Símbolo	Resto anterior	Conta	Resto novo
1	0	$(2 \times 0 + 1) \bmod 3 = 1$	1
0	1	$(2 \times 1 + 0) \bmod 3 = 2$	2
1	2	$(2 \times 2 + 1) \bmod 3 = 2$	2

Resto final = 2 $\neq$ 0. **Rejeita.**

String 1001 (vale 9 = 3 $\times$ 3):

$$S_0 \xrightarrow{1} S_1 \xrightarrow{0} S_2 \xrightarrow{0} S_1 \xrightarrow{1} S_0 \in F \quad \checkmark$$

Resto final = 0. **Aceita.**

10. EQUIVALÊNCIA E LIMITAÇÕES

↔ Equivalência com Expressões Regulares

Já vimos que um **DFA** é uma forma de descrever uma linguagem: ele aceita certas strings e rejeita outras. Mas existe outra forma de descrever linguagens: as **expressões regulares**.

Uma expressão regular é uma "fórmula" que descreve um **padrão** de strings. Por exemplo:

Expressão Regular	Significado	Strings geradas
01	exatamente 01	{01}
0*	zero ou mais 0 's	{ϵ, 0, 00, 000, ...}
(0 $\cup$ 1)*	qualquer sequência de 0 's e 1 's	{0, 1}*
1*01*01*	exatamente dois 0 's, com 1 's opcionais	{00, 100, 010, 001, ...}

↔ Notação das Expressões Regulares

As três operações básicas são:

Operação	Notação	Significado	Exemplo
Concatenação	ab	a seguido de b	01 gera {01}
União	$a \cup b$	a ou b	$0 \cup 1$ gera {0, 1}
Estrela (Kleene)	a^*	zero ou mais repetições de a	0^* gera $\{\epsilon, 0, 00, \dots\}$

Entendendo $1^*01^*01^*$ - leia da esquerda para a direita como uma receita:

$\underbrace{1^*}_{\text{zero ou mais 1's}} \underbrace{0}_{\text{um 0}} \underbrace{1^*}_{\text{zero ou mais 1's}} \underbrace{0}_{\text{um 0}} \underbrace{1^*}_{\text{zero ou mais 1's}}$

Ou seja: "1's opcionais, depois um 0, depois 1's opcionais, depois um 0, depois 1's opcionais".
Exatamente **dois 0's**, com **1's** podendo aparecer antes, entre ou depois deles.

Exemplos gerados: 00 (1^* vazio em todos), 100 ($1^* = 1$ no início), 010 ($1^* = 1$ no meio), 10011 ($1^* = 1, 0, 0, 1^* = 11$).

↔ DFAs e Expressões Regulares: Mesmo Poder

O resultado central é que **DFAs e expressões regulares descrevem exatamente as mesmas linguagens**:

- Toda linguagem reconhecida por um DFA pode ser escrita como expressão regular.
- Toda expressão regular pode ser convertida em um DFA.

A prova formal envolve as conversões $\text{DFA} \rightarrow \text{AFNG} \rightarrow \text{expressão regular}$ e $\text{expressão regular} \rightarrow \text{AFN} \rightarrow \text{DFA}$. Veremos isso em aulas futuras.

⚠ Limitações dos DFAs

DFAs são poderosos, mas **não podem reconhecer todas as linguagens**. A razão fundamental é que possuem **memória finita** (número fixo de estados).

Exemplos de linguagens que nenhum DFA reconhece:

- $L = \{a^n b^n \mid n \geq 0\}$: número igual de a 's seguidos de b 's.
- $L = \{0^n 1^n 0^n \mid n \geq 0\}$: três blocos iguais.
- $L = \{ww \mid w \in \{0, 1\}^*\}$: strings que são uma palavra repetida duas vezes.

Todas essas linguagens exigem **contar** ou **memorizar** partes da entrada para comparar com o que vem depois. Um DFA, com número fixo de estados, não consegue armazenar essa informação sem limite.

No decorrer do curso, veremos o **Lema do Bombeamento**, que fornece uma ferramenta formal para provar que certas linguagens **não** são regulares.

11. MINIMIZAÇÃO DE DFA_s

✚ Por que Minimizar?

Para uma mesma linguagem regular, podem existir **vários DFAs** que a reconhecem, com diferentes números de estados.

Pergunta natural: Existe um DFA com o **menor número de estados** possível?

Teorema: Para toda linguagem regular L , existe um **único** DFA mínimo (a menos de renomeação de estados) que aceita L .

Motivação prática: DFAs menores são mais eficientes em termos de memória e tempo de processamento.

Estados Equivalentes

A ideia central da minimização é identificar **estados equivalentes**, que podem ser "fundidos" em um só.

Definição: Dois estados p e q são **equivalentes** (ou indistinguíveis) se, para toda string $w \in \Sigma^*$:

$$\hat{\delta}(p, w) \in F \iff \hat{\delta}(q, w) \in F$$

Em outras palavras, p e q são equivalentes se **nenhuma string futura** consegue distingui-los: ambos aceitam ou ambos rejeitam as mesmas continuações.

Se p e q **não** são equivalentes, dizemos que são **distinguíveis**: existe alguma string w tal que um aceita e o outro rejeita.

⚙️ Algoritmo de Minimização (Ideia)

O algoritmo de minimização funciona por **refinamento de partições**:

1. Comece dividindo os estados em dois grupos: **finais** (F) e **não-finais** ($Q \setminus F$).
2. Para cada grupo, verifique se todos os estados do grupo se comportam da mesma forma (transições levam ao mesmo grupo). Se não, **divida** o grupo.
3. Repita até que nenhum grupo possa ser dividido.
4. O DFA mínimo é obtido usando cada **grupo final** como um único estado.

Este algoritmo é conhecido como **algoritmo de Hopcroft** e tem complexidade $O(n \log n)$, onde n é o número de estados.

12. RESUMO E EXERCÍCIOS

✓ Resumo da Aula

- **DFA:** 5-tupla $D = (Q, \Sigma, \delta, q_0, F)$, modelo de computação com estados finitos e transições determinísticas.
- **Representações equivalentes:** 5-tupla, diagrama de estados, tabela de transição.
- $\hat{\delta}$: extensão de δ para processar strings inteiras.
- **Aceitação:** w é aceita se $\hat{\delta}(q_0, w) \in F$.
- **Linguagem regular:** L tal que $L = L(D)$ para algum DFA D .
- **Fechamento:** linguagens regulares são fechadas sob união, interseção, complemento, concatenação e estrela de Kleene.
- **Equivalência:** DFAs e expressões regulares descrevem a mesma classe de linguagens.
- **Limitações:** DFAs não reconhecem linguagens como $\{a^n b^n\}$ (memória finita).
- **Minimização:** Todo DFA pode ser reduzido a um DFA mínimo único, identificando estados equivalentes.

❓ Verificação Rápida

Considere o DFA $A = (\{p, q, r\}, \{0, 1\}, \delta, p, \{r\})$ do Exercício Resolvido 3.1.

Determine se o DFA **aceita** ou **rejeita** cada string:

- 01
- 0
- 110
- 1001
- ε
- 0011

Respostas:

- 01: $p \rightarrow q \rightarrow r$ (**aceita**)
- 0: $p \rightarrow q$ (**rejeita**)
- 110: $p \rightarrow p \rightarrow p \rightarrow q$ (**rejeita**)
- 1001: $p \rightarrow p \rightarrow q \rightarrow q \rightarrow r$ (**aceita**)
- ε : fica em p (**rejeita**)
- 0011: $p \rightarrow q \rightarrow q \rightarrow r \rightarrow r$ (**aceita**)

Exercícios

Exercício 3.1 Projete uma máquina de vender chocolates que custe R\$ 2,00, aceitando moedas de R\$ 0,10, R\$ 0,50 e R\$ 1,00. Desenhe o diagrama e apresente a definição formal.

Exercício 3.2 Seja $\Sigma = \{a, b, c\}$. Construa um DFA para a linguagem $L = \{w \in \Sigma^* \mid w \text{ é da forma } xbbya, \text{ onde } x, y \in \Sigma^*\}$.

Exercício 3.3 Seja $\Sigma = \{0, 1\}$. Construa DFAs para:

- (a) $L = \{w \mid w \text{ tem número ímpar de 1's}\}$
- (b) $L = \{w \mid |w| \leq 3\}$
- (c) $L = \{w \mid w \text{ tem número par de 0's e número par de 1's}\}$
- (d) $L = \{w \mid w \text{ é terminada em } 00\}$
- (e) $L = \{w \mid w \text{ contém a substring } 011\}$

Exercícios (cont.)

Exercício 3.4 Seja $D = (Q, \Sigma, \delta, q_0, F)$ um DFA. Sejam $x, y \in \Sigma^*$, $a \in \Sigma$ e $q \in Q$.

(a) Mostre que $\hat{\delta}(q, xy) = \hat{\delta}(\hat{\delta}(q, x), y)$.

(b) Mostre que $\hat{\delta}(q, ax) = \hat{\delta}(\delta(q, a), x)$.

Exercício 3.5 Considere $D = (Q, \Sigma, \delta, q_0, F)$ com:

- $Q = \{q_0, q_1, \dots, q_6\}$, $\Sigma = \{0, 1, 2, \dots, 6\}$, $F = \{q_0\}$
- $\delta(q_j, i) = q_k$, sendo $k = (j + i) \bmod 7$

Qual é a linguagem aceita por D ?

Exercício 3.6 Prove que se L é regular, então $\overline{L}$ também é regular.

Exercício 3.7 Forneça um DFA com $\Sigma = \{0, 1\}$ que aceite $L_3 = \{w \mid N(w) \text{ é múltiplo de } 3\}$.